Introduction to Embedded Systems

EE 186 – Lecture 7

Analog Signals, ADC, DAC

Zerina Kapetanovic

Today we are learning about…

- Overview of Analog Signals in Embedded Systems
  - Sampling Signals
  - Data Conversion

- Analog to Digital Converters (ADC)
- Digital to Analog Converters (DAC)

Reading

- **STM RM0432 Reference Manual**
  - Section 21: Analog to Digital Converter (ADC)

- **Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (Third Edition)**
  - Chapter 20: Analog to Digital Converter (ADC)

The physical world is … analog

- **Digital systems need to…**
  - Measure analog quantities (sensor inputs, sound waveforms, etc.)
  - Control analog systems (actuators, sound waveforms, etc.)
- **Analog-to-digital (ADC)**
  - Audio recording
- **Digital-to-analog (DAC)**
  - Audio playback

Analog Signals Overview

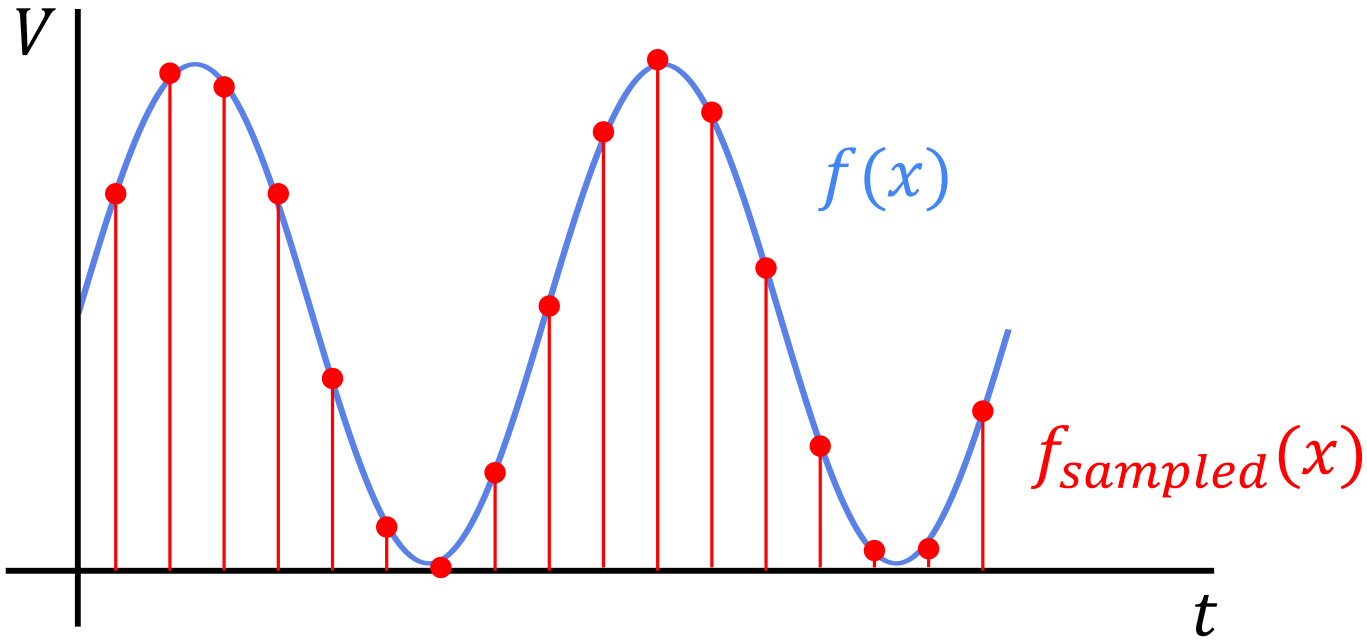
- An **analog signal** is one that is continuous in both amplitude and time
- To represent a signal in the digital domain **we must approximate** it in two ways:
  - Amplitude quantizing
  - Time quantizing

converting continuous or large sets of values into a discrete set of values.

- The **range** of the system is the maximum minus the minimum value
- The **precision** of the system defines the number of values from which the amplitude of the digital signal is selected
- The **resolution** is the smallest change in value that is significant

Analog Signal in the Digital Domain

How do we represent an analog signal?



Sampling

The sampling rate is the rate at which amplitude values are digitized from the original waveform

Can you observe the sampling rate?

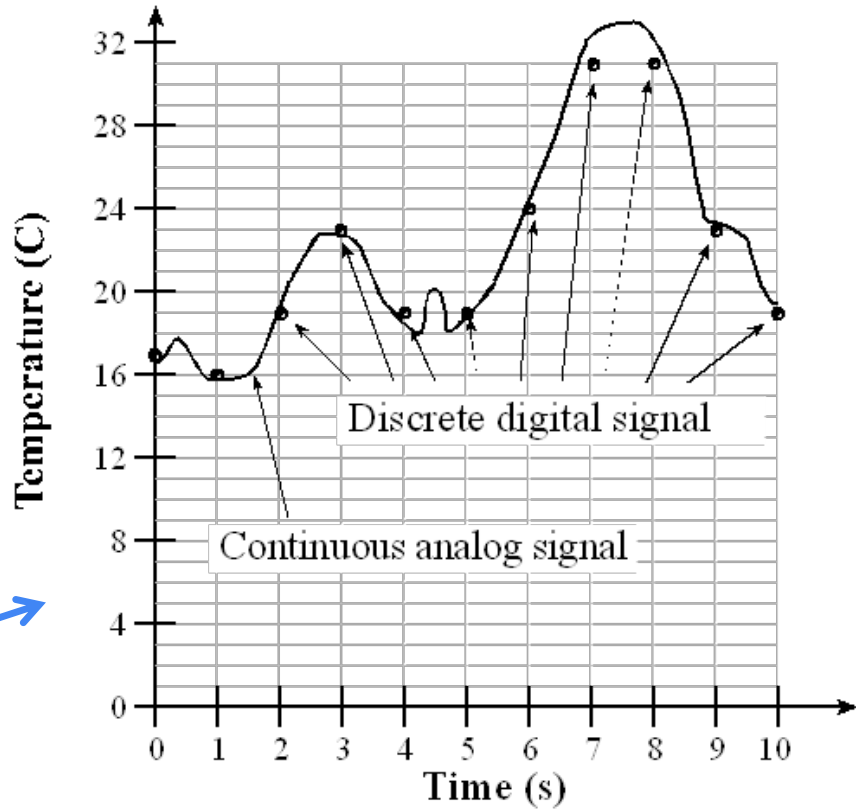


Photo Credit: Jonathan Volvano (UT Austin)

Shannon-Nyquist Theorem

If a continuous-time signal $f(x)$ contains **no frequencies higher than f_{max}**, it can be completely determined by discrete samples taken at a rate:

$$f_{samples} > 2f_{max}$$

Example

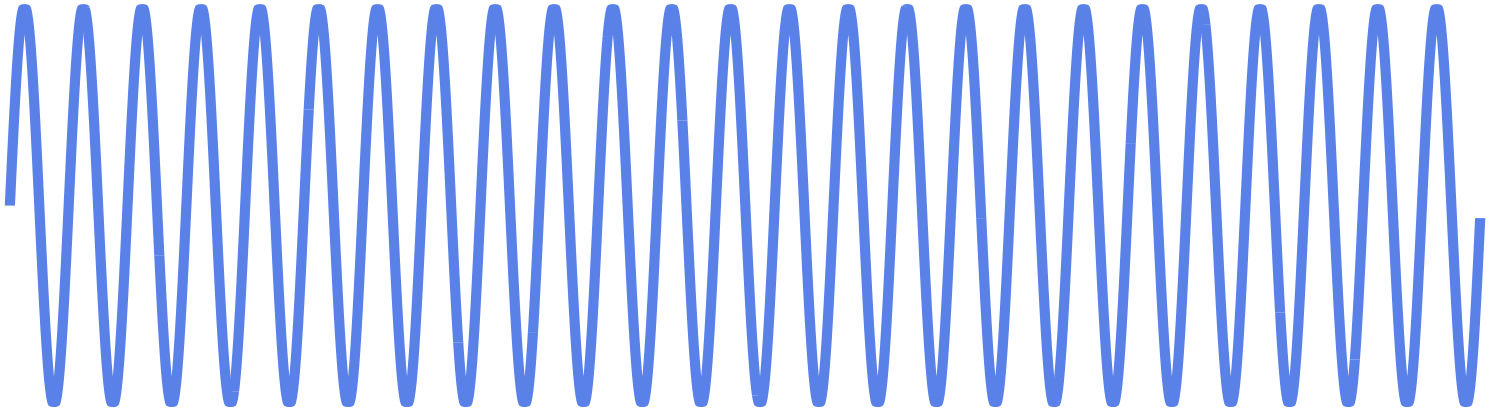
Humans can process audio signals
20Hz – 20kHz

Audio CDs: Sampled at 44.1 kHz



Example: Sampling a signal

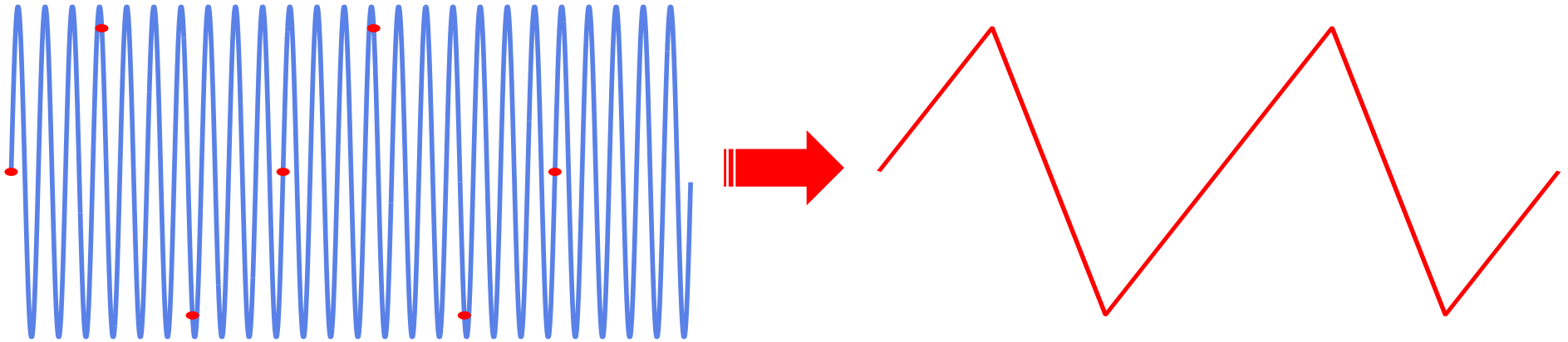
$f(t) = \sin(2\pi f_c t)\,, f_c = 5Hz$



Example: Sampling a signal

$$f(t) = \sin(2\pi f_c t), \quad f_c = 5Hz$$

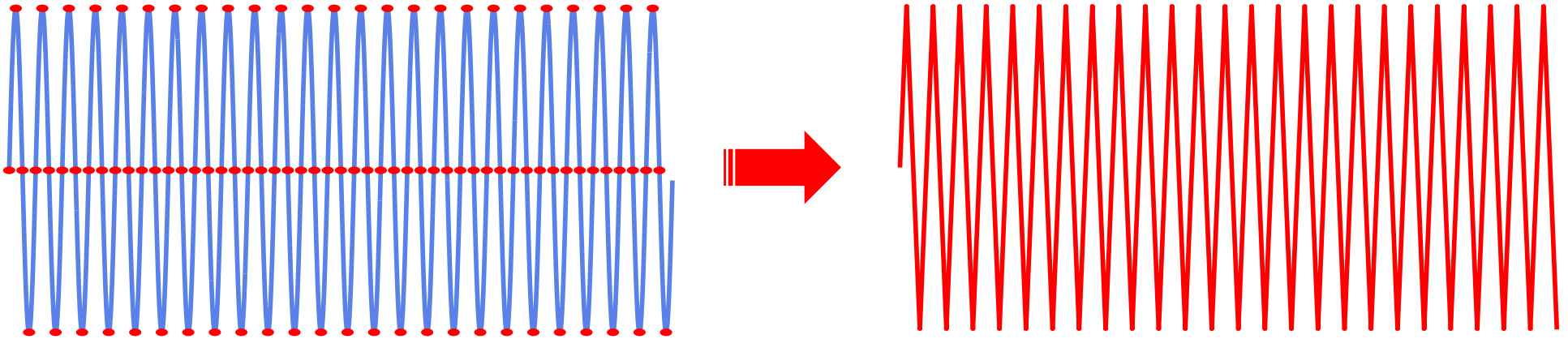
What signal do I expect to see if $f_s = 1.5Hz$?



Example: Sampling a signal

$f(t) = \sin(2\pi f_c t)\,, \quad f_c = 5Hz$

What signal do I expect to see if $f_s = 20Hz$?



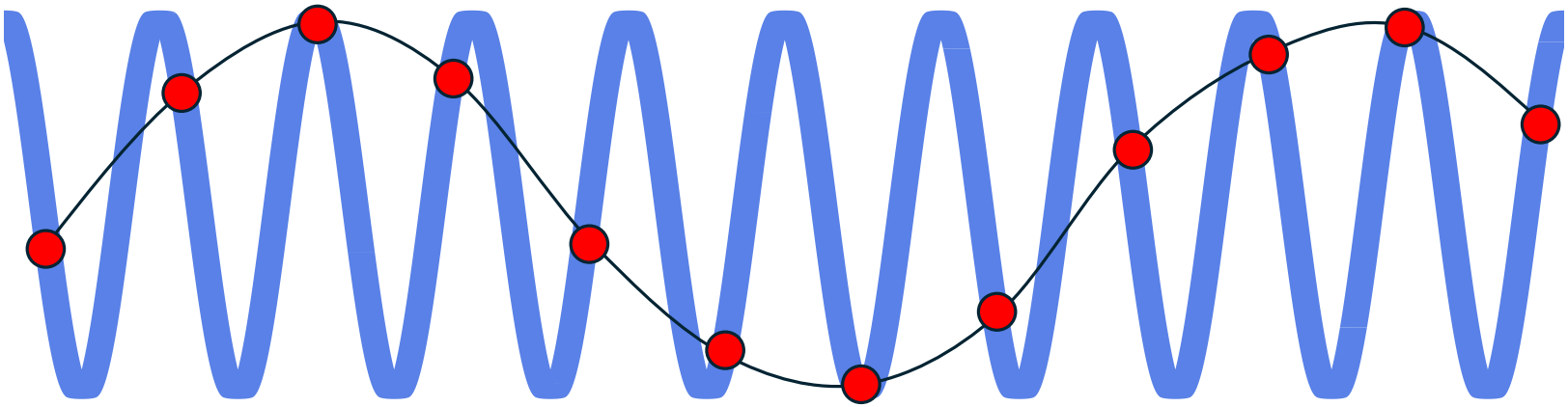
Shannon-Nyquist Theorem

If a continuous-time signal $f(x)$ contains **no frequencies higher than f_{max}**, it can be completely determined by discrete samples taken at a rate:

$$f_{samples} > 2f_{max}$$

Aliasing

Aliasing occurs **when a continuous signal is sampled at a rate insufficient to capture its changes accurately.** This results in the observance of false frequencies, or aliases, that were not present in the original signal[1].



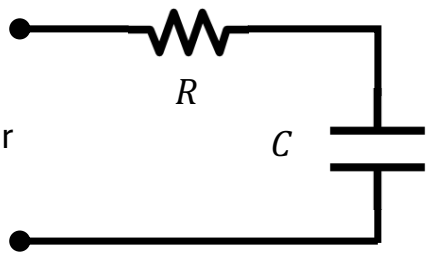
[1] Keysight Technologies. (n.d.). What is aliasing? Keysight Knowledge. https://www.keysight.com/used/us/en/knowledge/glossary/oscilloscopes/what-is-aliasing

Anti-aliasing Filter

A low-pass filter **passes lower frequencies** and **attenuates higher frequencies**

$$|V_{out}| = |V_{in}| \frac{1}{\sqrt{1 + \omega^2 R^2 C^2}}$$

$$\omega = 2\pi f$$



RC Low Pass Filter

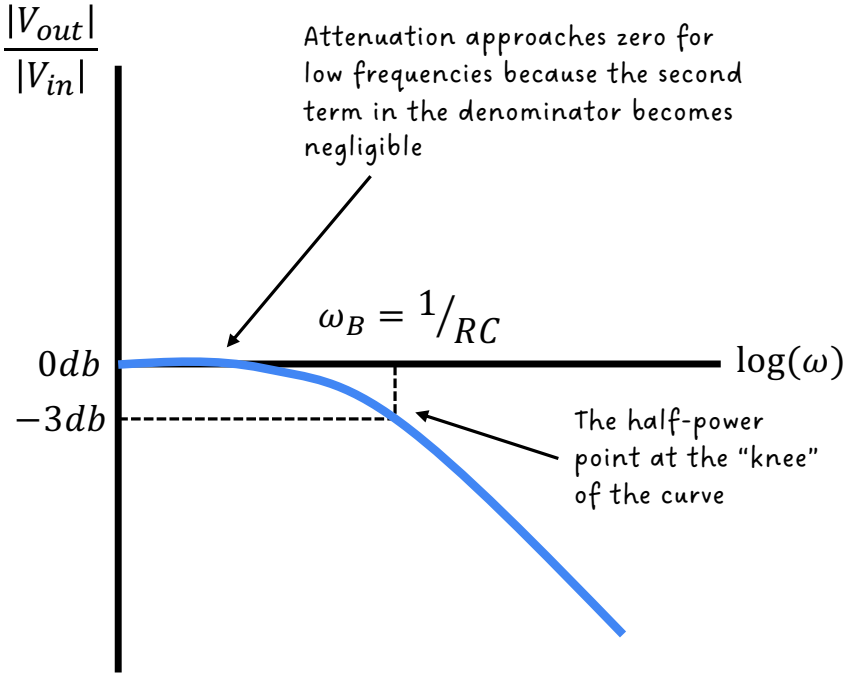


Figure adapted from Falstad

Group Exercise

Find R and C values so that the half-power point occurs at 30Hz

$$|V_{out}| = |V_{in}| \frac{1}{\sqrt{1 + \omega^2 R^2 C^2}}$$

ω *is in radians*, $\omega = 2\pi f$

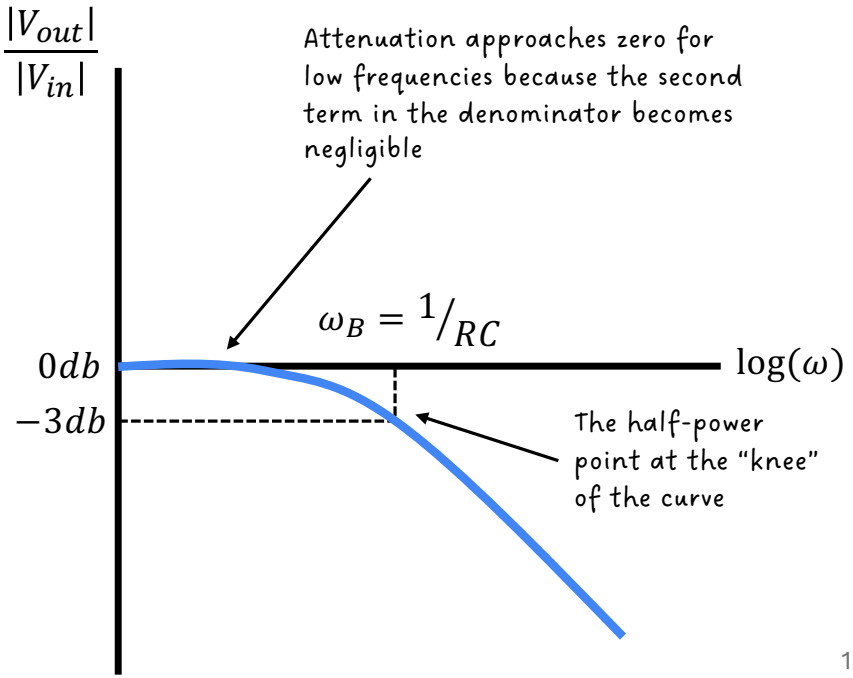


Figure adapted from Falstad

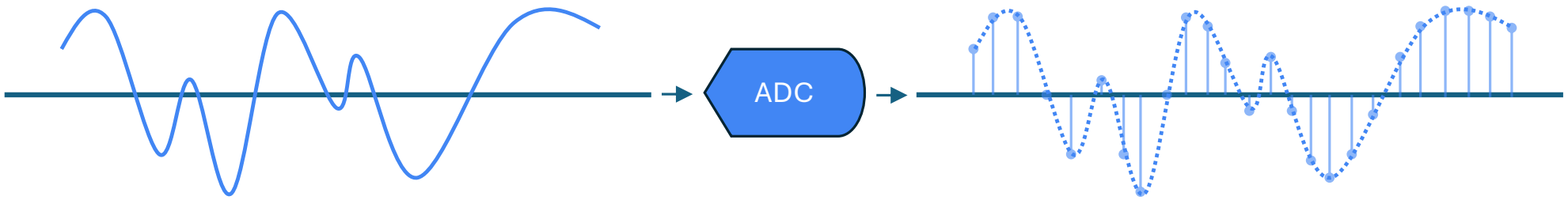
Analog to Digital Converter (ADC)

An analog-to-digital converter (ADC) converts an analog signal into digital form

We use ADCs in embedded systems to collect information about the external world

The input signal is usually analog voltage and the output is a binary number

Analog to Digital Conversion (ADC)



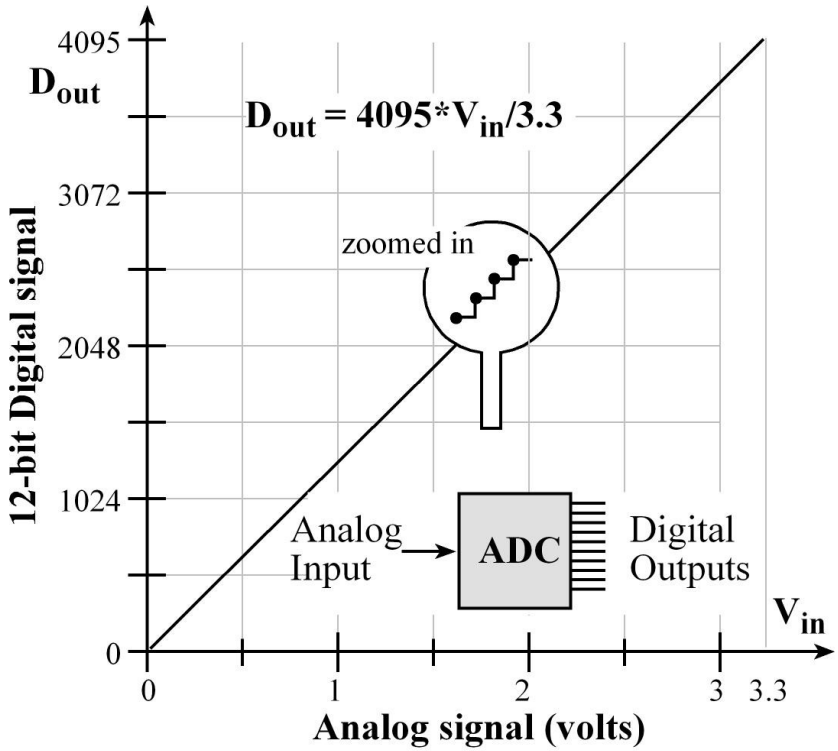
Continuous-time analog signal

Discrete-time digital values

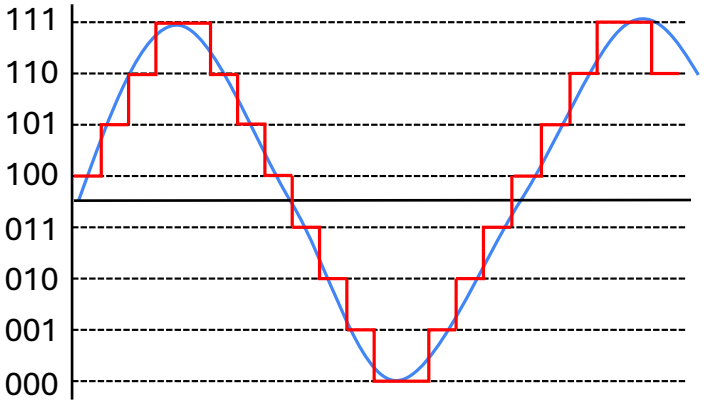
Analog to Digital Converter (ADC)

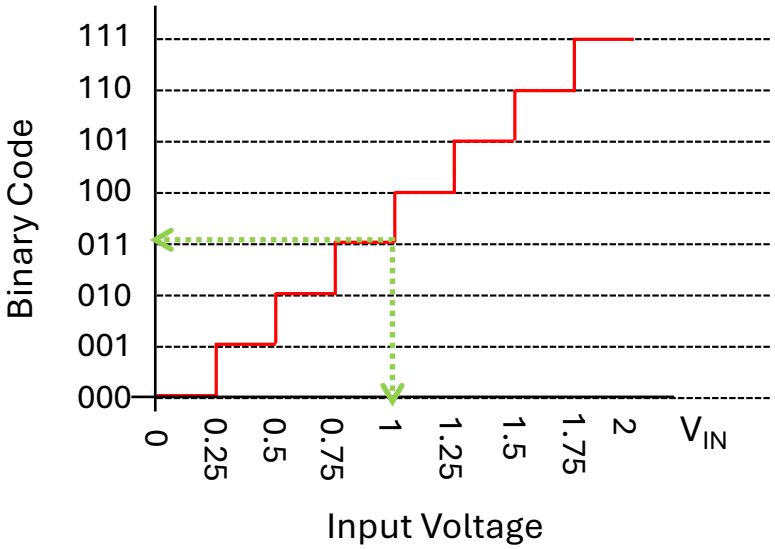
Range is the maximum and minimum ADC input (e.g., 0 to +3.3V)

Resolution is the smallest distinguishable change in the input (e.g., 3.3V/4095 … about 0.81mV)



Discretization





Resolution is determined by the number of bits (in binary) used to represent an analog input

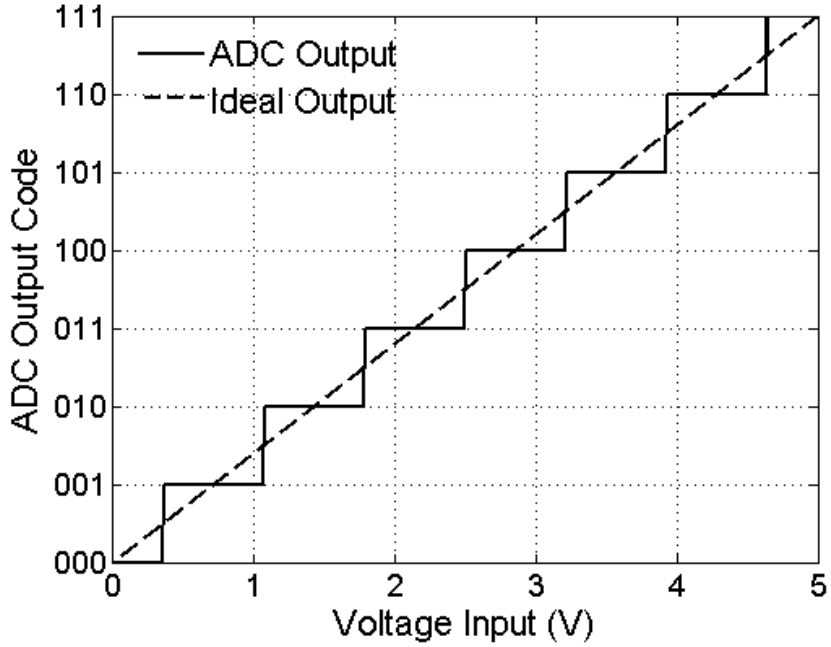
Quantization Method

If the ADC output has *n* bits, the ADC result of an input voltage *V* can be calculated as follows:

$$Digital_{floor} = round\left((2^n - 1) \times \frac{V - V_{RL}}{V_{RH} - V_{RL}}\right)$$

High reference voltage

Low reference voltage



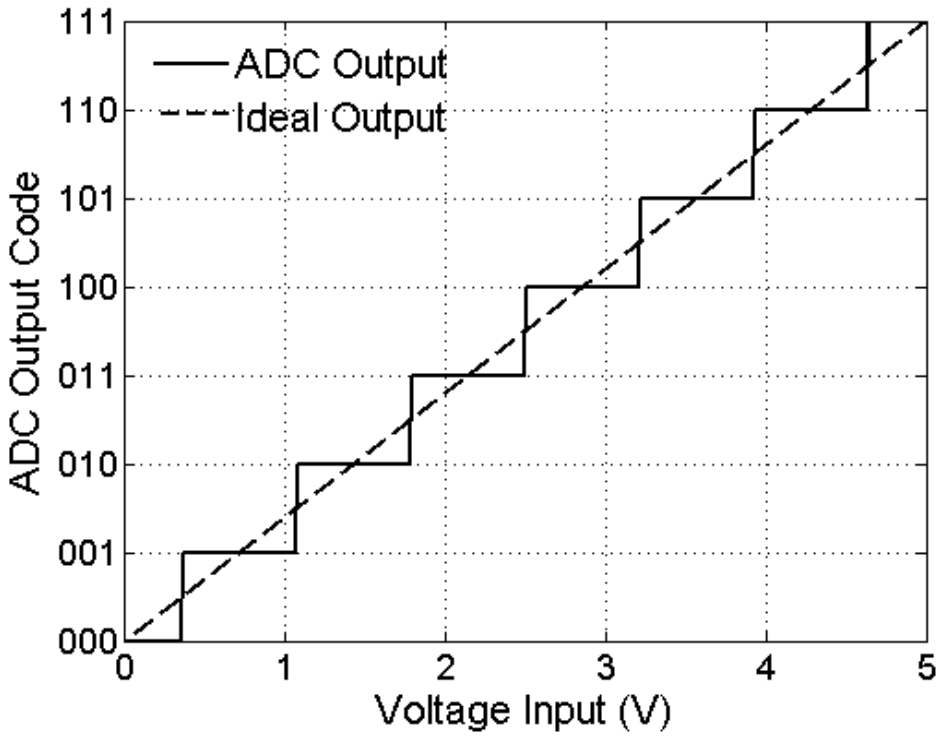
Quantization Methods

Single Ended Mode

$$Digital_{floor} = round\left((2^n - 1) \times \frac{V}{V_{RH}}\right)$$

$V_{REF} = 5V \qquad n = 3$

What is the ADC output if the voltage input is 2.2V?



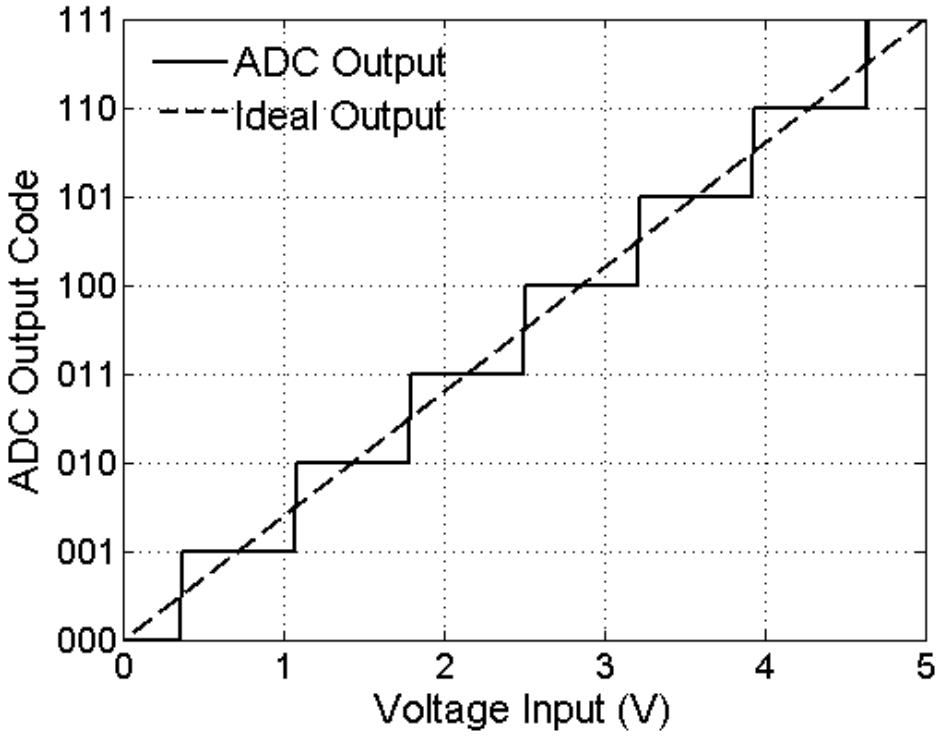
Quantization Error

Resolution is determined by the number of bits (in binary) used to represent an analog input

Larger step size → less information

Quantization error is how far off the discrete value is from actual

Larger step size → larger error



Quantization Error

Step Size $(\Delta) = \frac{V_{REF}}{2^n}$

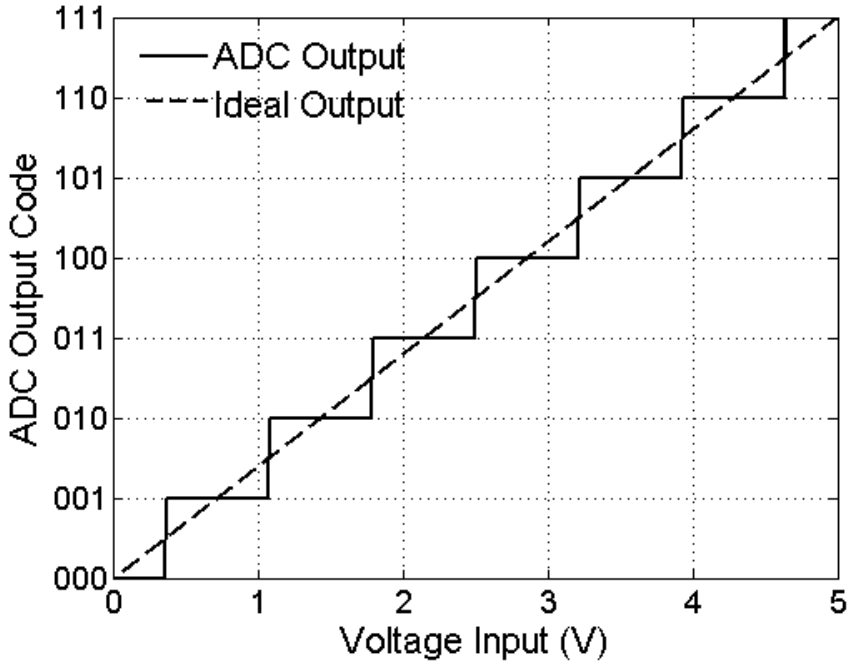
$(\Delta) = \frac{5}{8} = 0.625\ V$

$Actual = 2.2V \qquad ADC\ Output = 3V$

$V_{Quantized} = ADC\ Output \times \Delta = 3 \times 0.625 = 1.875V$

$Quantization\ Error = 2.2\ - 1.875 = 0.325V$

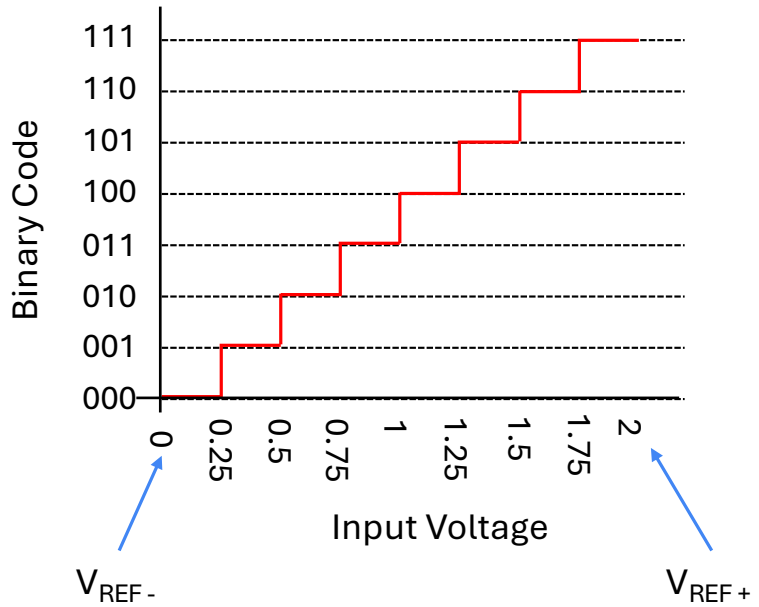
$\frac{1}{2}\Delta$



Transfer Function

Convert the ADC binary output code to a voltage value

$$V_{code} = ADC\ Output \times \frac{V_{REF}}{2^n}$$



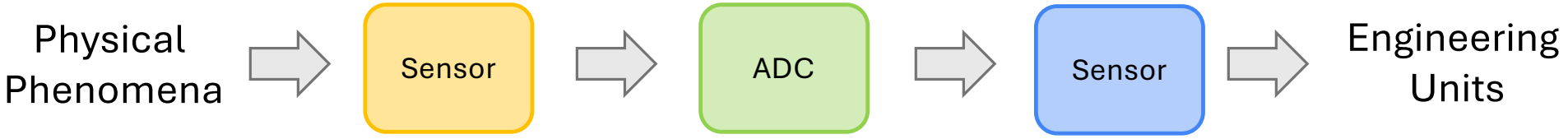
Data Conversion Example

Analog to Digital – How do we get there?

What we would like:



How do we actually do it:



Example: Analog Sensor

From the datasheet!

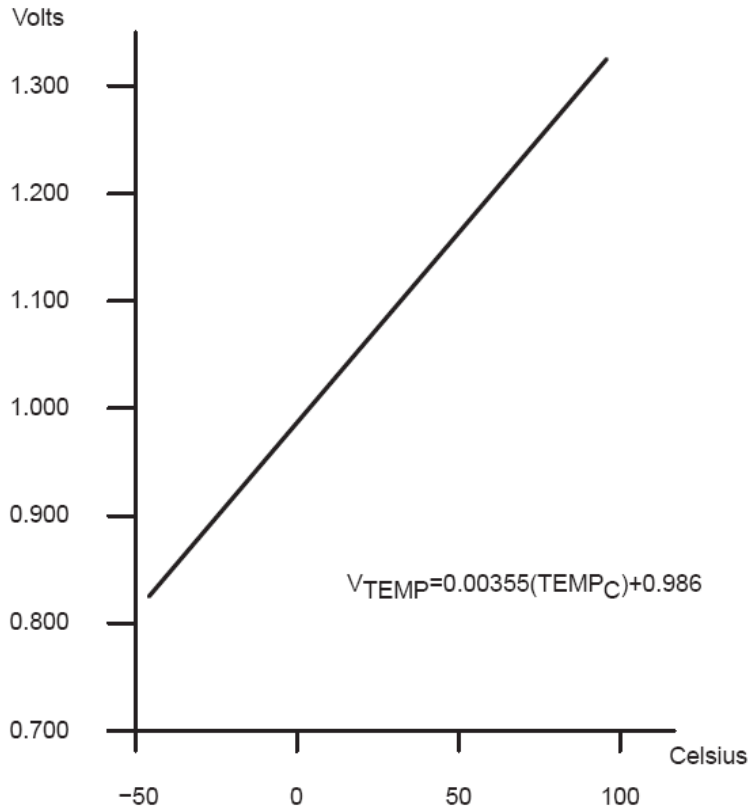
Analog Temperature Sensor



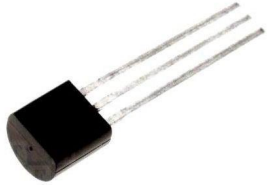
Analog Soil Moisture Sensor



Output curve and transfer function



Group Exercise



Three Pin Analog Temperature Sensor

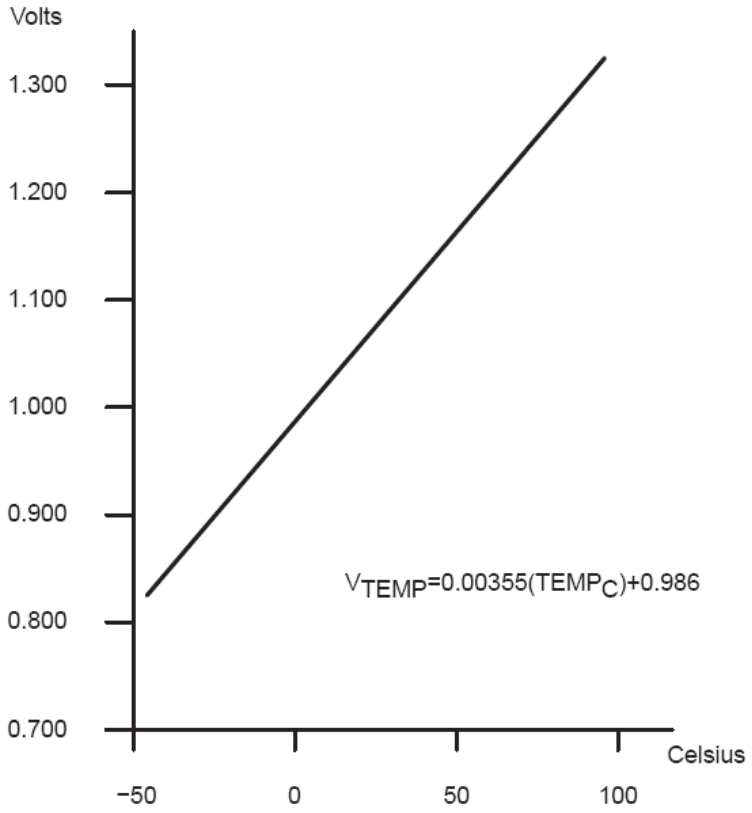
- Assume that the maximum valid output voltage for this sensor is 1.4 V
- We are using an MCU with a 10-bit ADC, referenced from 0 to 3.3V

What is the *minimum* step size of this ADC?

What is the transfer function you will need to write in your code? (ADC Value --> Temperature)

What is the ADC value at 50C

Output curve and transfer function



Analog to Digital Converter (ADC)

- Types of ADCs
  - Flash
  - Single Slope Integration
  - Successive-Approximation Register
  - STML4R5 ADC
- Sample and Hold Circuit

Recap: Analog to Digital Converter (ADC)

- An ADC produces a finite-precision digital number to represent approximately the size of an analog voltage relative to a reference voltage
- Key performance parameters:
  - Sampling rate
  - Resolution
  - Power Dissipation

ADC Architecture

There are three popular ADC architectures

Sigma-Delta

- Low-speed applications
  - Voice/audio
- Low-sampling rates
- Typically < 100ksps

Successive-Approximation (SAR)

- Low-power data acquisition
- Moderate sampling rates
- Typically < 5Msps

Pipelined

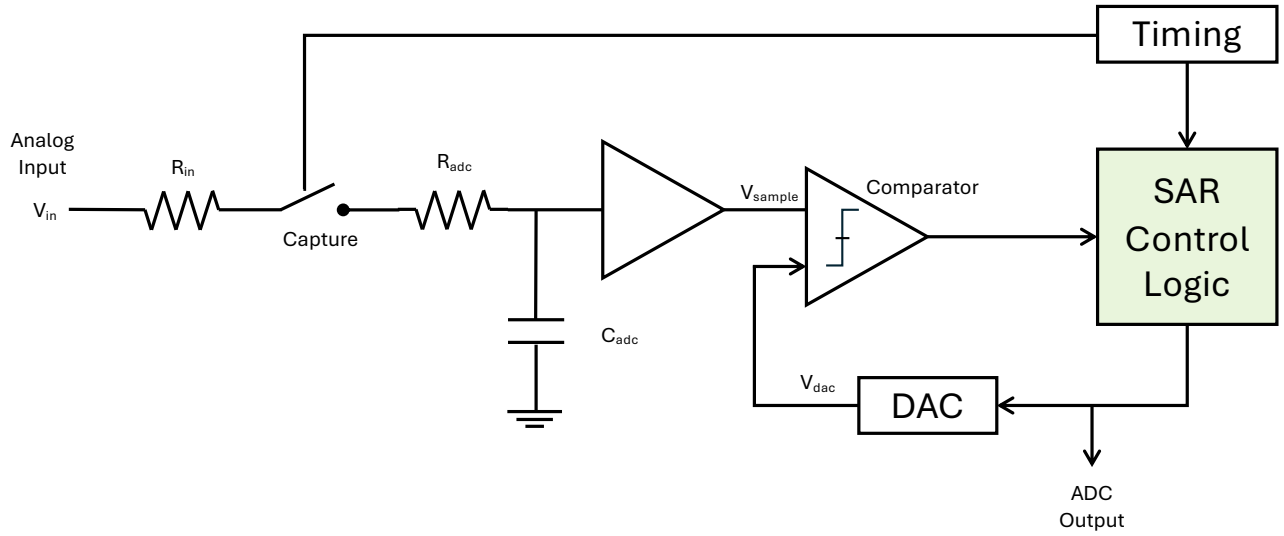
- High-speed applications
  - Digital oscilloscopes
  - Radar Communication
- Fast sampling
- Typically > 5Msps

The ADC on STM32 Microcontrollers is based on the *successive-approximation* (SAR) architecture

SAR Architecture

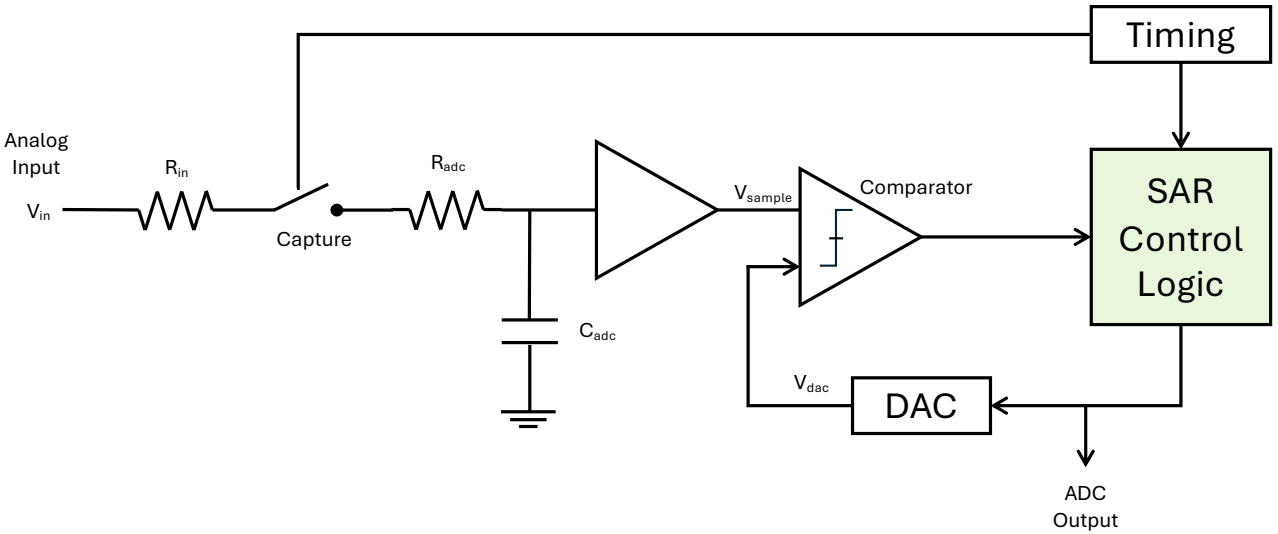
Two main components:

- SAR digital quantization
- Sample and hold amplifier



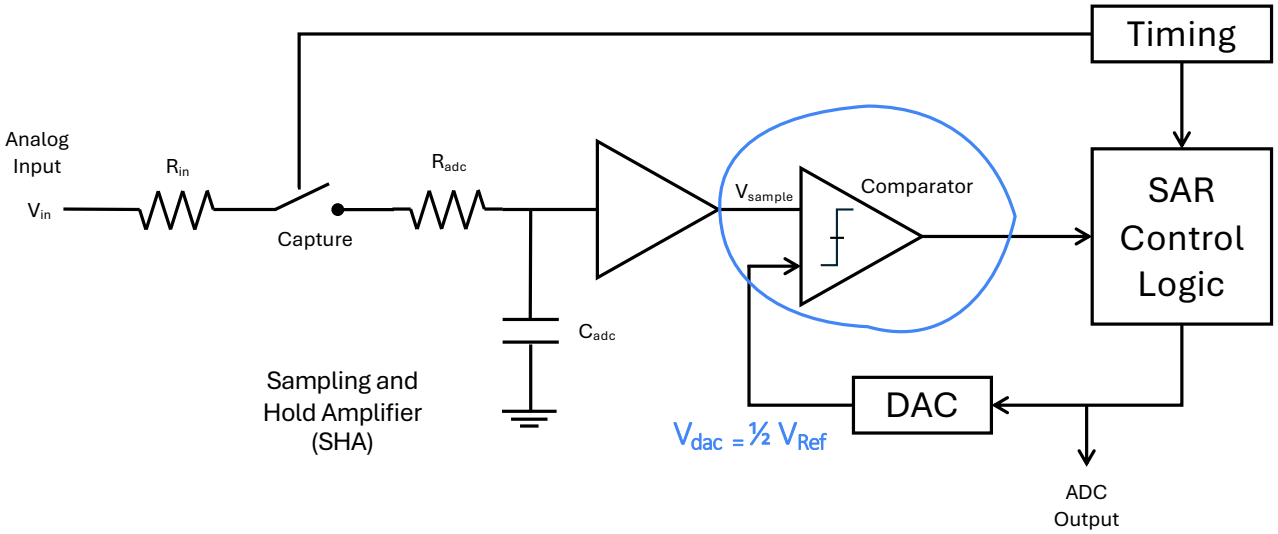
SAR Digital Quantization

SAR Digital Quantization



- Use the binary search algorithm to find the digital number (ADC output) that represents the analog input most closely
- Dynamically changes the ADC output so that the DAC output gradually approaches the DAC input voltage

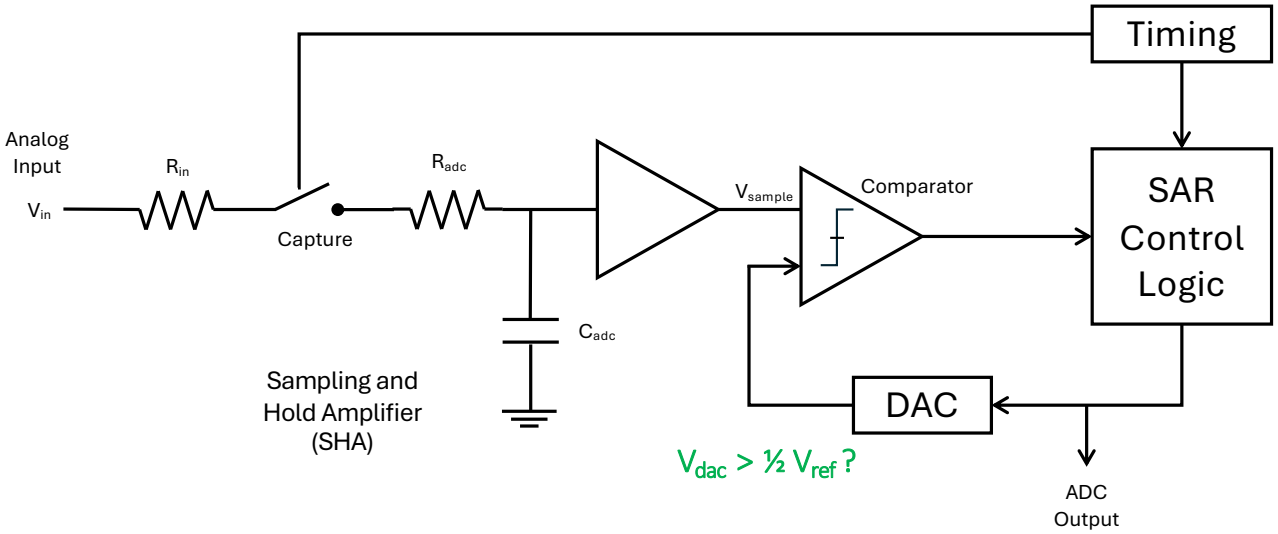
SAR Digital Quantization



1

Set the internal DAC output V_{dac} to ½ V_{Ref} and compare V_{dac} with the ADC input V_{in}

SAR Digital Quantization



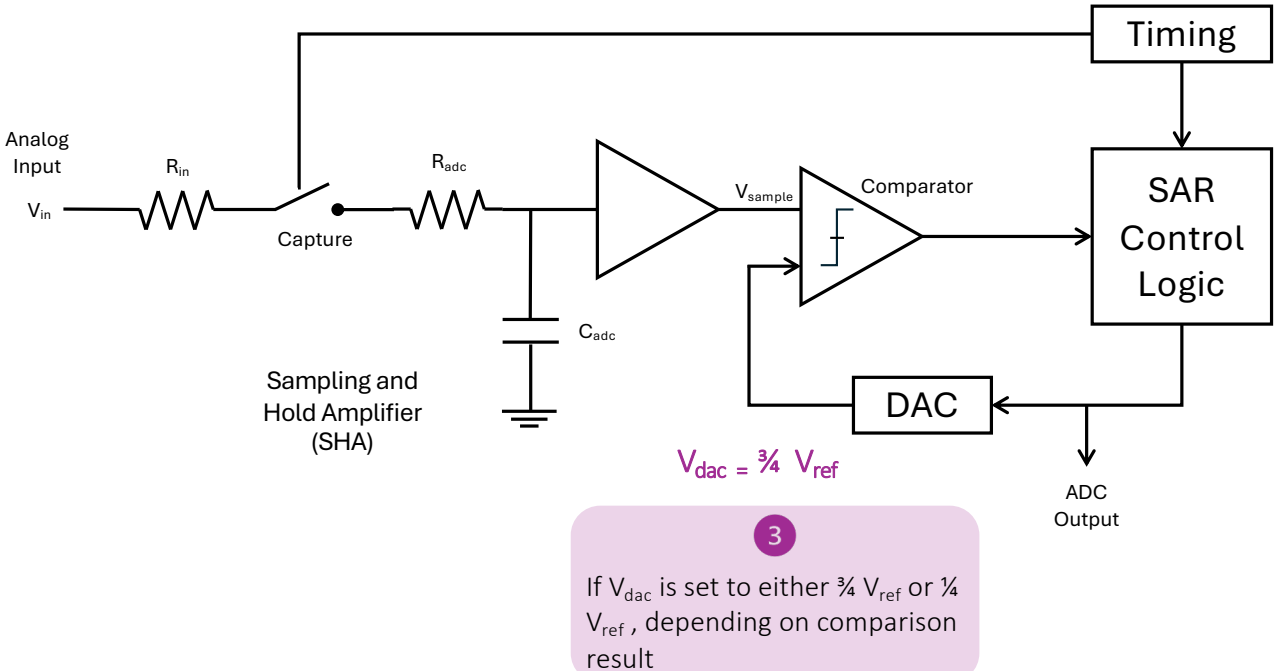
1

Set the internal DAC output V_{dac} to ½ V_{Ref} and compare V_{dac} with the ADC input V_{in}

2

If V_{in} is larger than ½ V_{ref} , set the most significant bit (MSB) of the ADC result. Otherwise, clear the MSB of the ADC

SAR Digital Quantization



1

Set the internal DAC output V_{dac} to ½ V_{Ref} and compare V_{dac} with the ADC input V_{in}

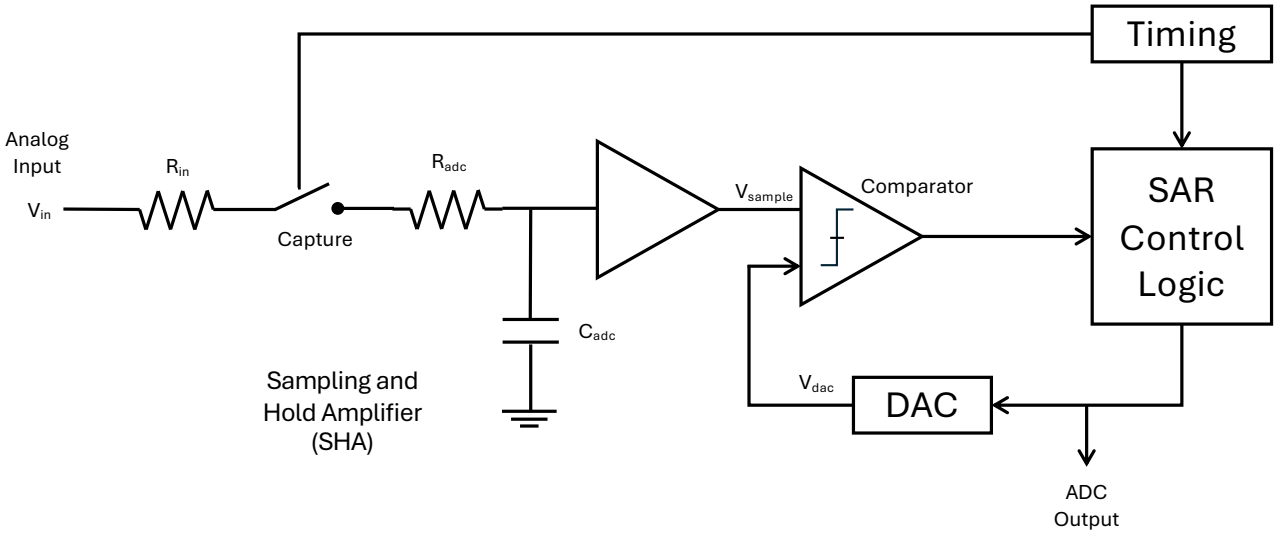
2

If V_{in} is larger than ½ V_{ref} , set the most significant bit (MSB) of the ADC result. Otherwise, clear the MSB of the ADC

3

If V_{dac} is set to either ¾ V_{ref} or ¼ V_{ref} , depending on comparison result

SAR Digital Quantization

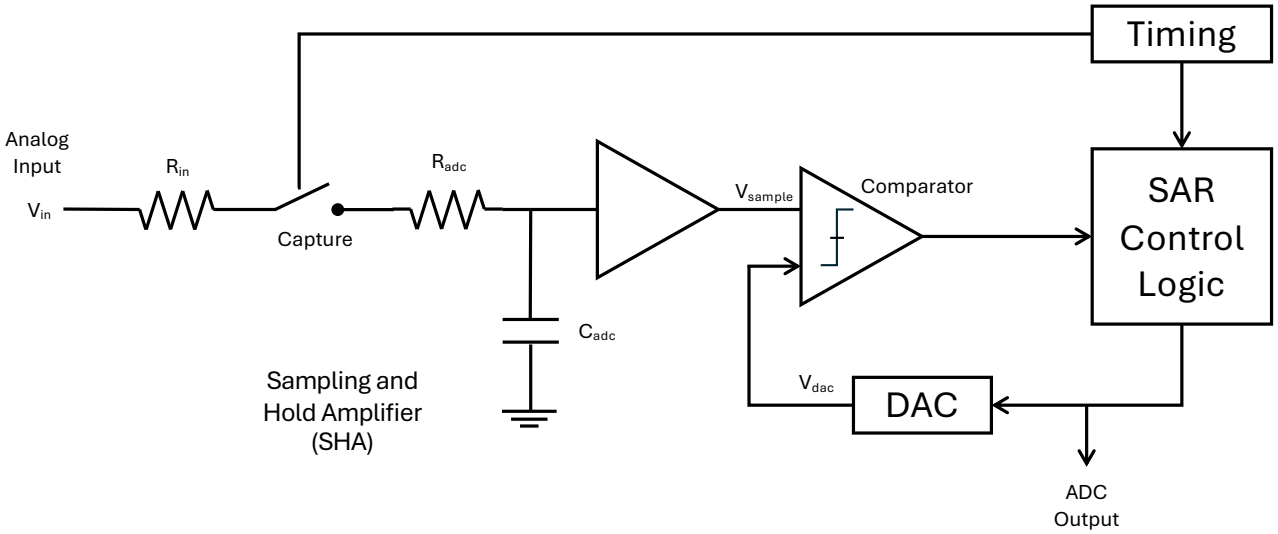


1. Set the internal DAC output V_{dac} to ½ V_{Ref} and compare V_{dac} with the ADC input V_{in}

2. If V_{in} is larger than ½ V_{ref}, set the most significant bit (MSB) of the ADC result. Otherwise, clear the MSB of the ADC

3. If V_{dac} is set to either ¾ V_{ref} or ¼ V_{ref}, depending on comparison result

SAR Digital Quantization



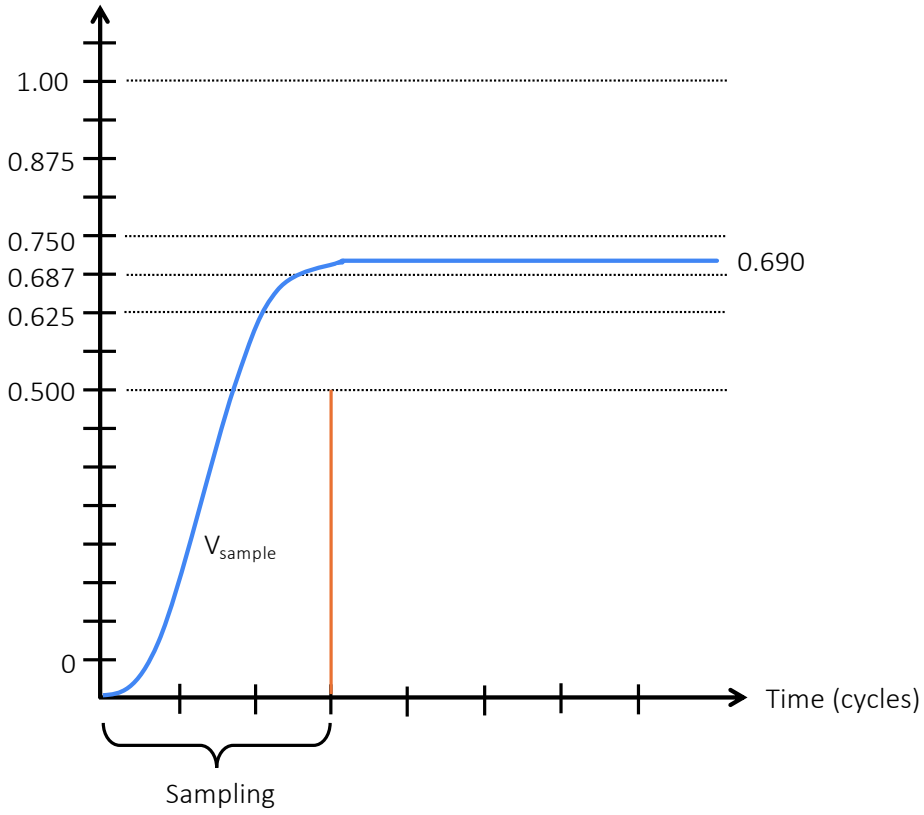
If the ADC has a resolution of *n* bits, the successive-approximation conversion takes *n* steps to complete

1. Set the internal DAC output V_{dac} to ½ V_{Ref} and compare V_{dac} with the ADC input V_{in}
2. If V_{in} is larger than ½ V_{ref}, set the most significant bit (MSB) of the ADC result. Otherwise, clear the MSB of the ADC
3. If V_{dac} is set to either ¾ V_{ref} or ¼ V_{ref}, depending on comparison result
4. Repeat until all bits of the ADC output have been determined

Example: SAR Digital Quantization

Input Voltage Range: 0 to 1V

After the input voltage is sampled,
$V_{dac} = ½ V_{ref} = 0.5V$

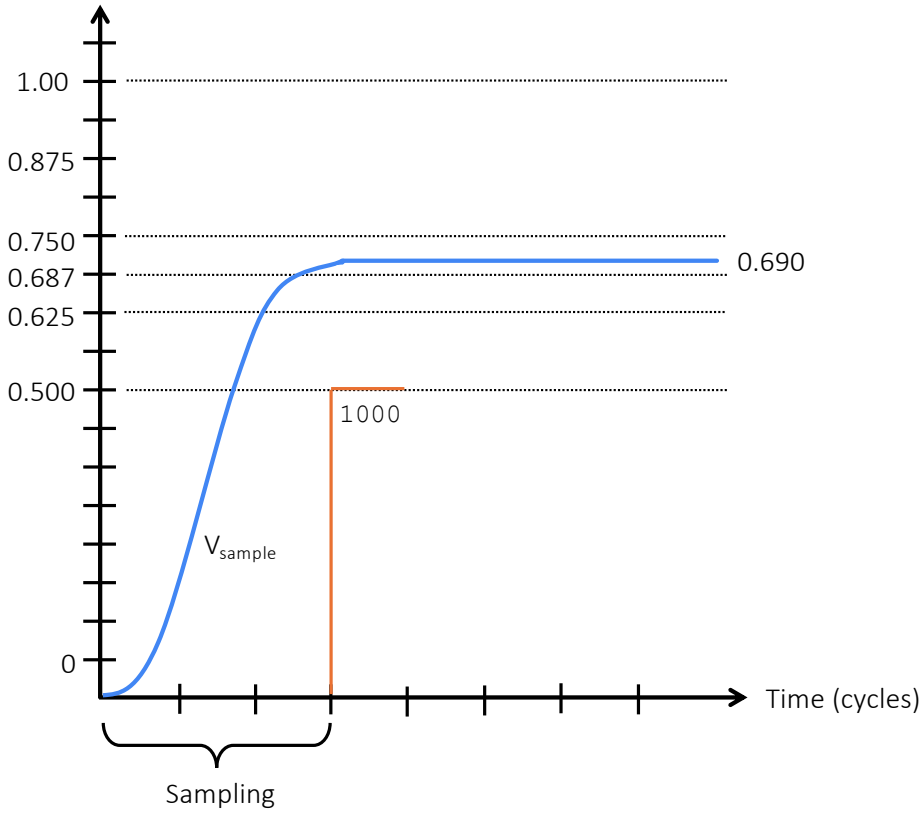


Example: SAR Digital Quantization

Input Voltage Range: 0 to 1V

After the input voltage is sampled,
$V_{dac} = ½ V_{ref} = 0.5V$

V_{sample} is **larger than** 0.5 so the MSB is set to 1



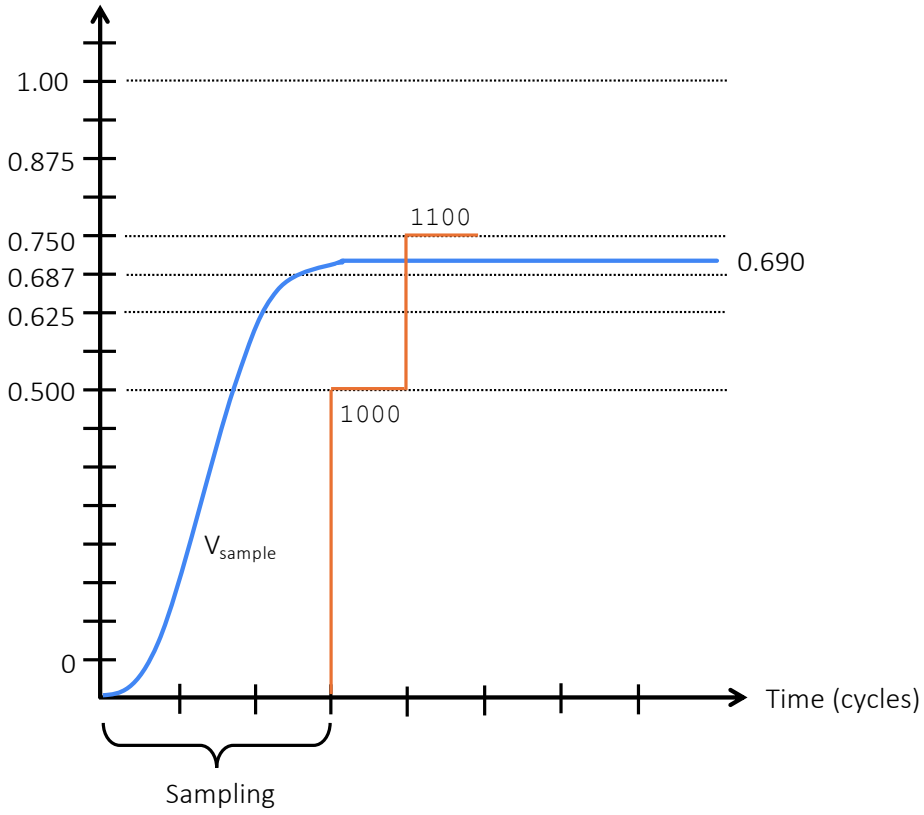
Example: SAR Digital Quantization

Input Voltage Range: 0 to 1V

After the input voltage is sampled,
$V_{dac} = ½ V_{ref} = 0.5V$

V_{sample} is **larger than** 0.5 so the MSB is set to 1

V_{dac} is set to 0.750V (¾ V_{ref}), which sets the ADC ouput to `0b110`



Example: SAR Digital Quantization

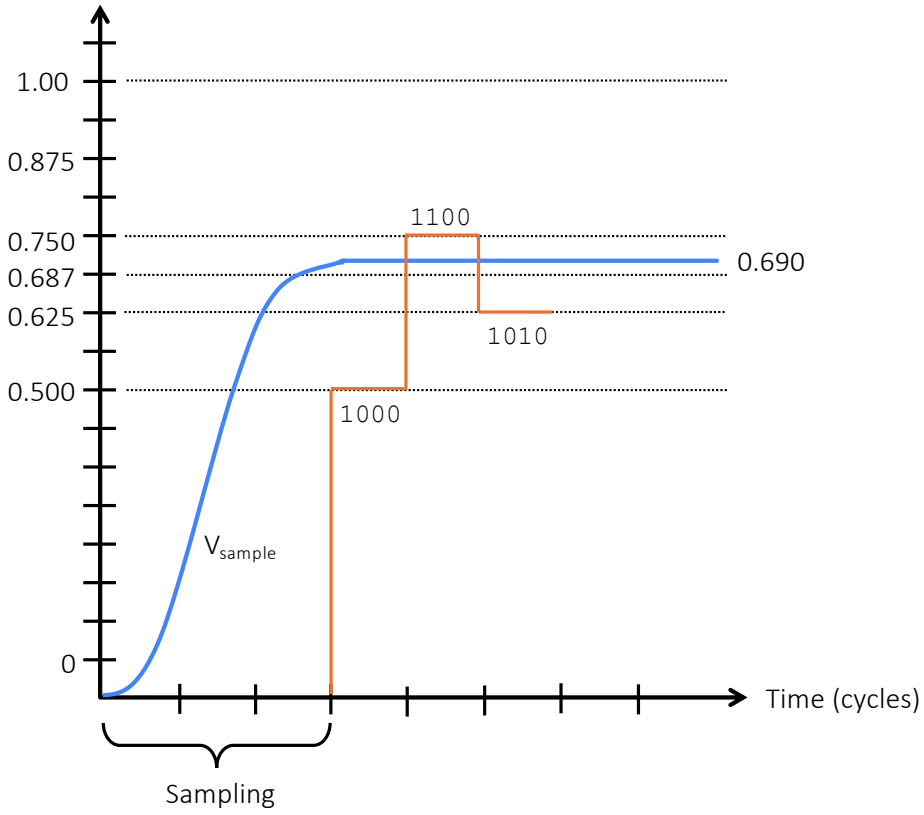
Input Voltage Range: 0 to 1V

After the input voltage is sampled,
$V_{dac} = ½ V_{ref} = 0.5V$

V_{sample} is **larger than** 0.5 so the MSB is set to 1

V_{dac} is set to 0.750V (¾ V_{ref}), which sets the ADC ouput to `0b110`

V_{sample} is **less than** 0.750V, so the the control logic sets **V_{dac} to 0.625V**



Example: SAR Digital Quantization

Input Voltage Range: 0 to 1V

After the input voltage is sampled,
$V_{dac} = ½ V_{ref} = 0.5V$

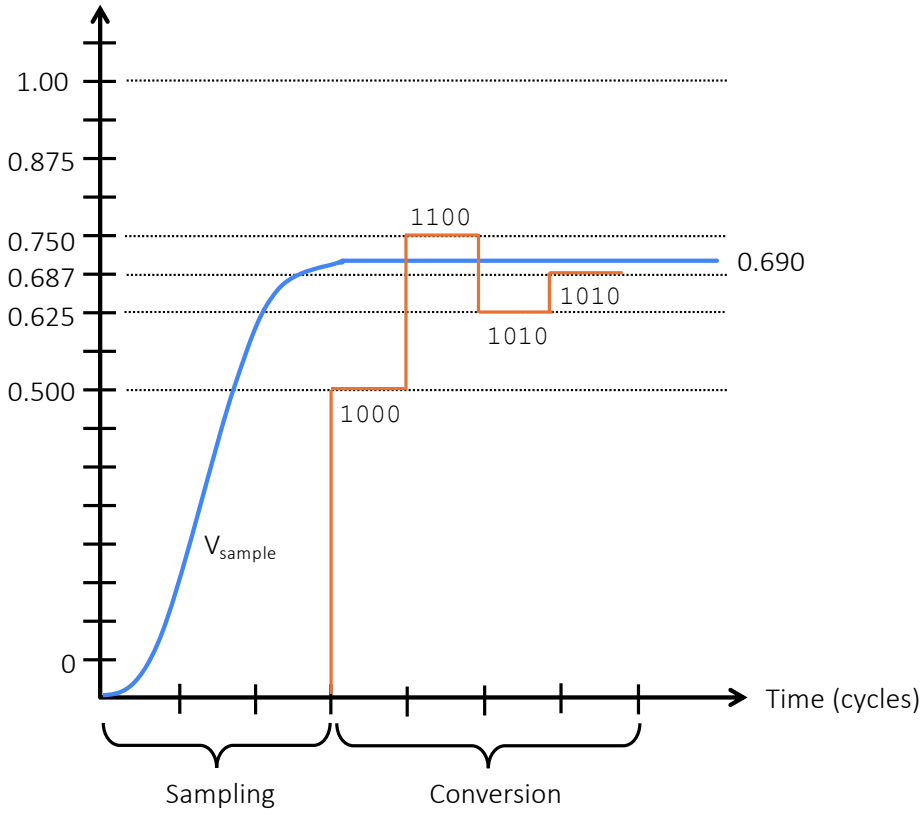
V_{sample} is **larger than** 0.5 so the MSB is set to 1

V_{dac} is set to 0.750V (¾ V_{ref}), which sets the ADC output to `0b110`

V_{sample} is **less than** 0.750V, so the the control logic sets **V_{dac} to 0.625V**

⋮

The **final ADC output is `0b1011`**, and the conversion process took four cycles

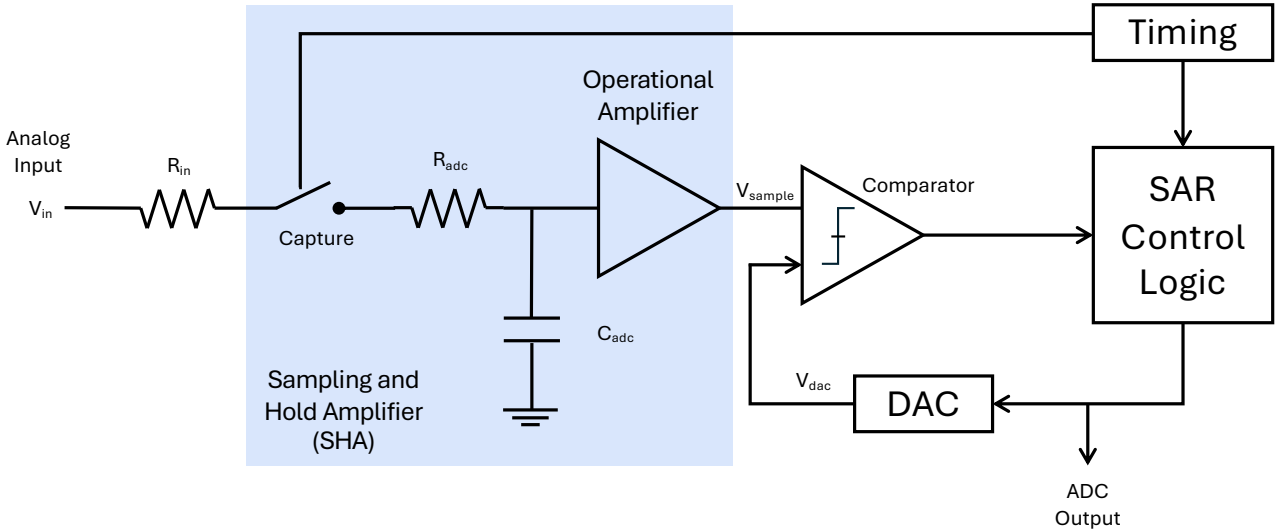


Sample and Hold Circuit

Sample and Hold

$$V_c(t) = V_{in} \times \left(1 - e^{t/T_c}\right)$$

$$T_c = (R_{in} + R_{adc}) \times C_{adc}$$



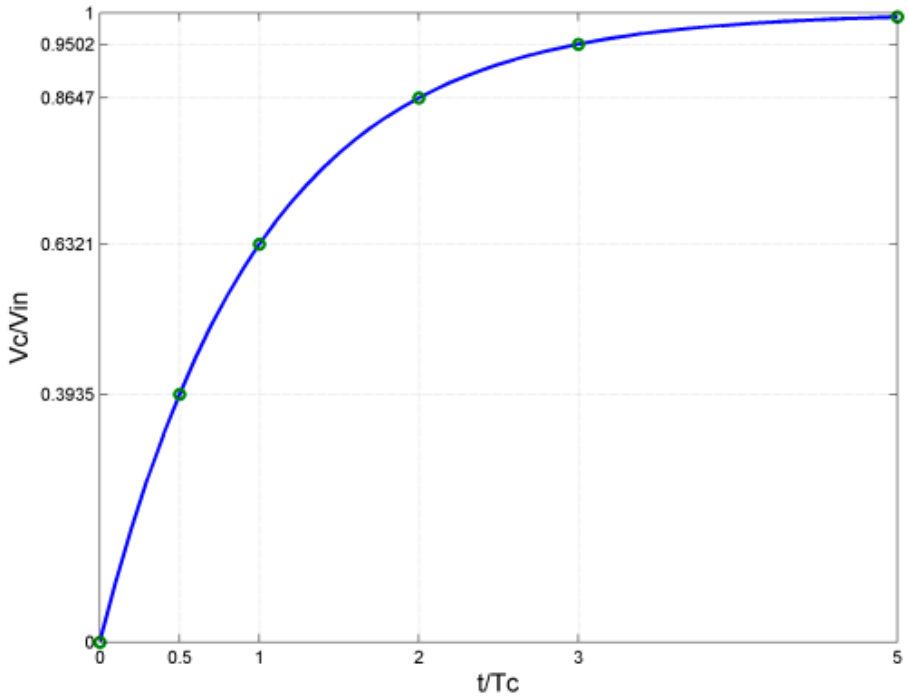
Sample and Hold

$V_c(t) = V_{in} \times \left(1 - e^{t/T_c}\right)$

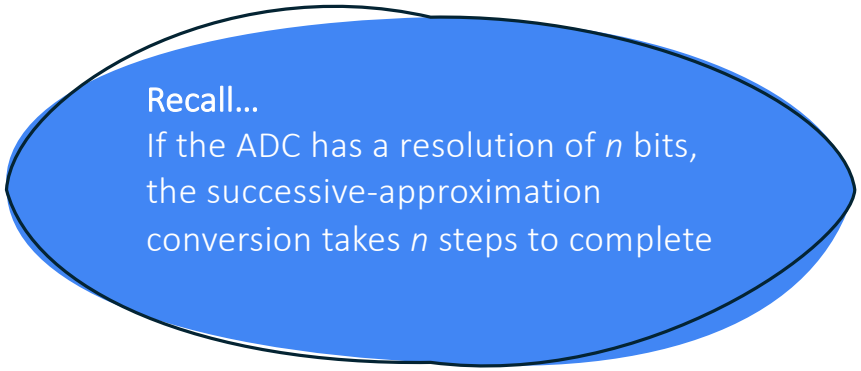
$T_c = (R_{in} + R_{adc}) \times C_{adc}$

V_{in} cannot be sampled instantly

Example: if the switch is closed for a time period 3Tc, the voltage across the capacitor is only 95.02% of the input voltage



ADC Conversion Time

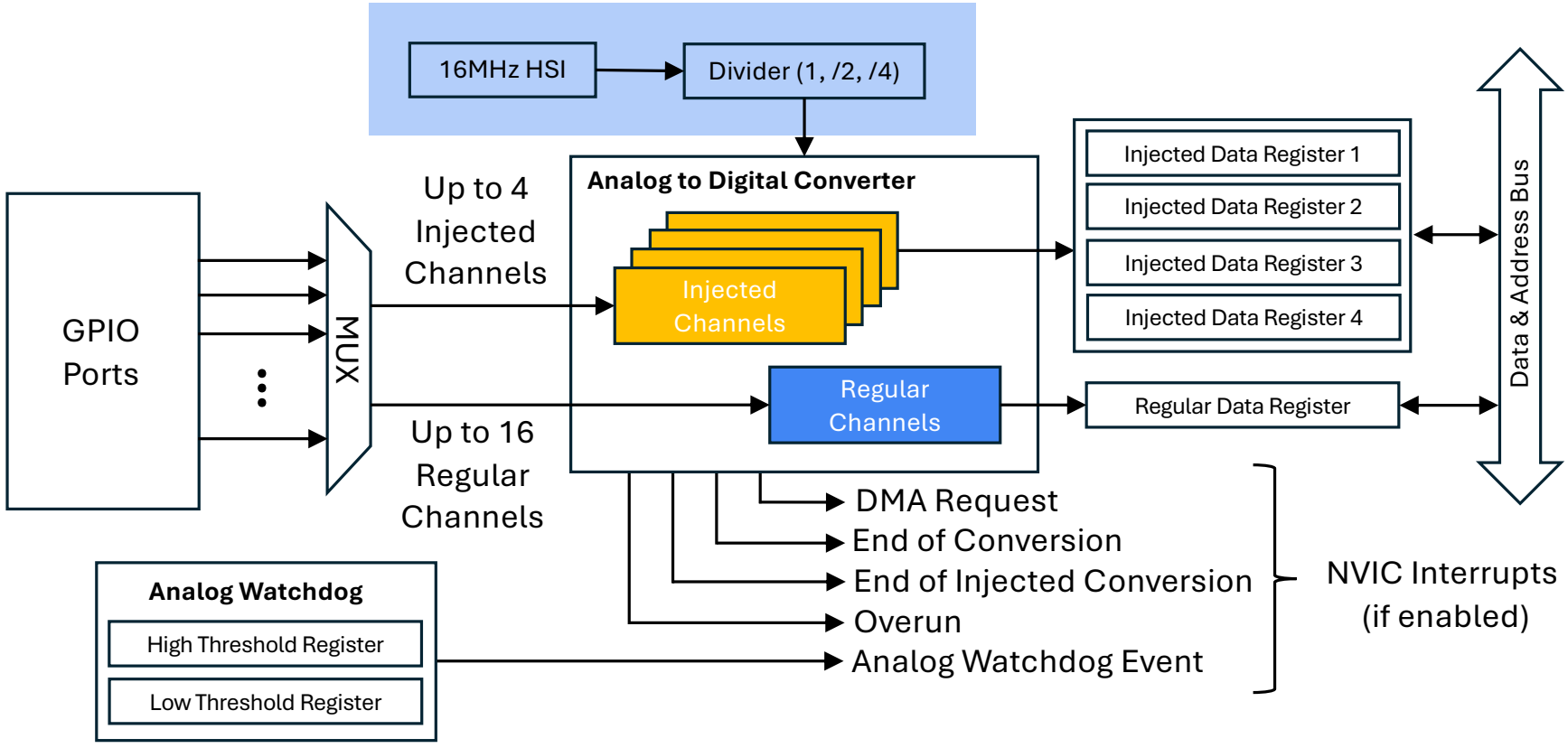


$T_{ADC} = T_{sampling} + T_{conversion}$

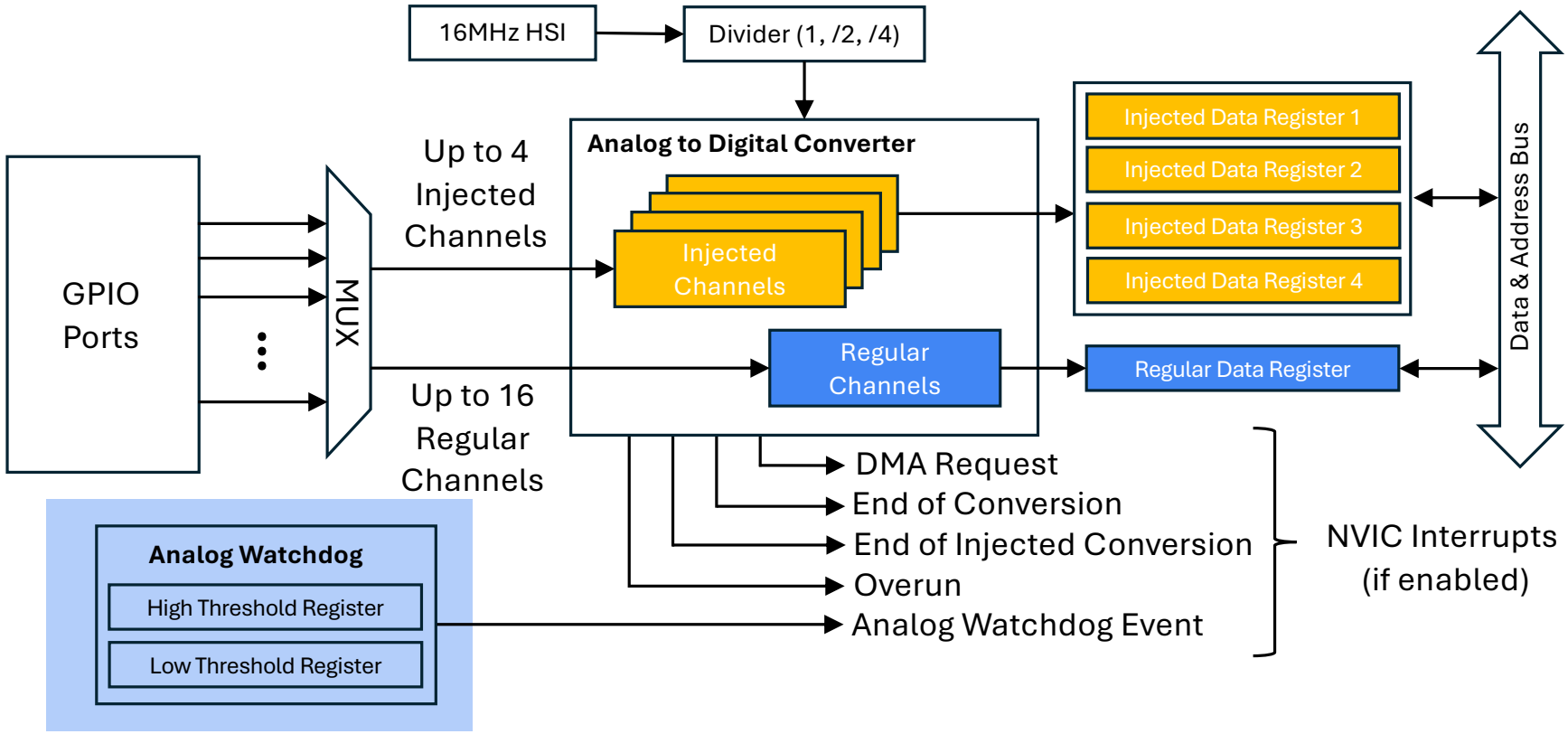
$T_{sampling}$ is software configurable

For a 12-bit ADC what is the conversion time of the **sampling time is set to 4 cycles** and the **ADC clock is set to 16MHz**?

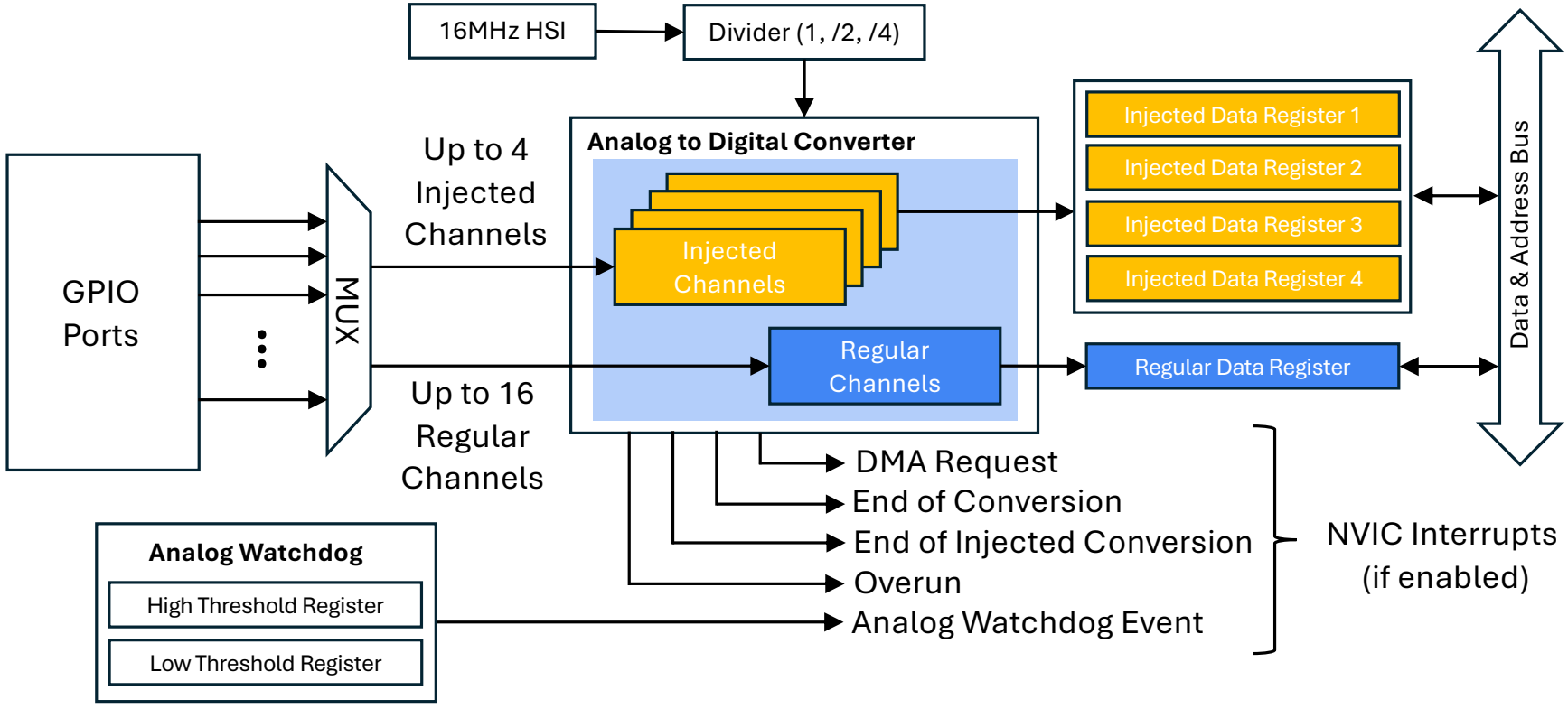
STM32L4 ADC


16MHz HSI
Divider (1, /2, /4)
Analog to Digital Converter
GPIO Ports
MUX
Up to 4 Injected Channels
Injected Channels
Regular Channels
Up to 16 Regular Channels
Injected Data Register 1
Injected Data Register 2
Injected Data Register 3
Injected Data Register 4
Regular Data Register
Data & Address Bus
DMA Request
End of Conversion
End of Injected Conversion
Overun
Analog Watchdog Event
NVIC Interrupts (if enabled)
Analog Watchdog
High Threshold Register
Low Threshold Register

STM32L4 ADC


16MHz HSI
Divider (1, /2, /4)
Analog to Digital Converter
Injected Data Register 1
Injected Data Register 2
Injected Data Register 3
Injected Data Register 4
Data & Address Bus
GPIO Ports
MUX
Up to 4 Injected Channels
Injected Channels
Regular Channels
Regular Data Register
Up to 16 Regular Channels
DMA Request
End of Conversion
End of Injected Conversion
Overun
Analog Watchdog Event
NVIC Interrupts (if enabled)
Analog Watchdog
High Threshold Register
Low Threshold Register

STM32L4 ADC


16MHz HSI
Divider (1, /2, /4)
Analog to Digital Converter
Injected Data Register 1
Injected Data Register 2
Injected Data Register 3
Injected Data Register 4
Data & Address Bus
GPIO Ports
MUX
Up to 4 Injected Channels
Injected Channels
Regular Channels
Regular Data Register
Up to 16 Regular Channels
DMA Request
End of Conversion
End of Injected Conversion
Overun
Analog Watchdog Event
NVIC Interrupts (if enabled)
Analog Watchdog
High Threshold Register
Low Threshold Register

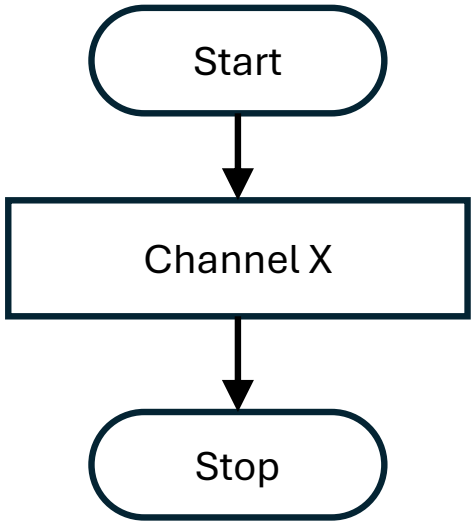
ADC Conversion Modes

The ADC module performs conversions on selected channels in either **single-conversion mode** or **continuous-conversion mode**.

ADC Conversion Mode

Single Conversion On One Input Channel (Regular Group)

1. The target channel is determined by the SQ1[4:0] bits in the SQR1 register
2. The 16-bit ADC data register (ADC_DR) holds the conversion result
3. After the conversion, the *end of regular conversion flag* (EOC) is set in the ISR register.
4. The ADC module generates an interrupt request if the EOCIE bit is set in the interrupt enable register (IER)

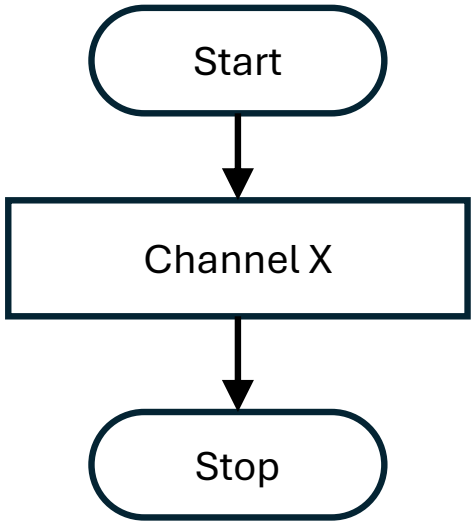


ADC Conversion Mode

Single Conversion On One Input Channel

(Injected Group)

1. The JSQ1[4:0] bits in the JSQR register selects conversion channel
2. The 16-bit register (ADC_JDR1) holds the conversion result
3. After the conversion, the *end of injected conversion flag* (EOC) is set in the ISR register.
4. The ADC module generates an interrupt request if the JEOCIE bit is set in the interrupt enable register (IER)

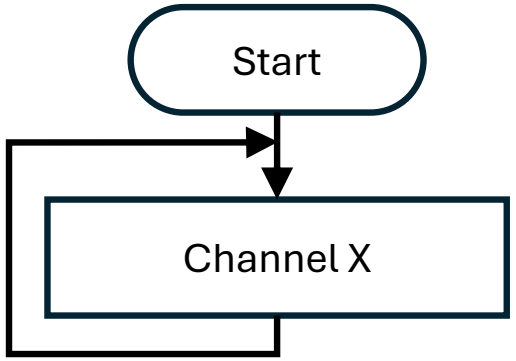


ADC Conversion Mode

Single Channel, Continuous Conversion

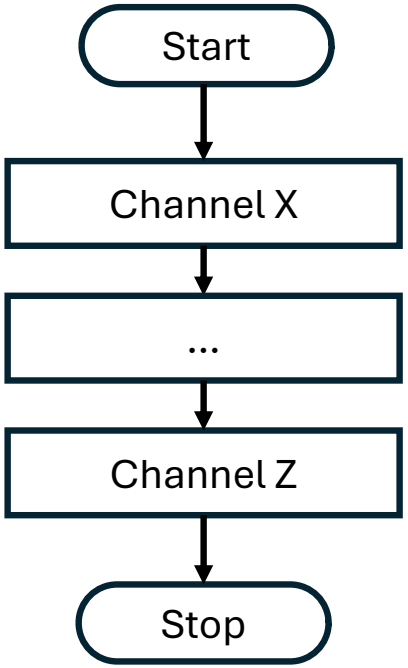
- ADC module automatically starts a new conversion immediately after it finishes one.

- The last conversion result is saved in the ADC_DR register for a regular channel and ADC_JDR1 for an injected channel.

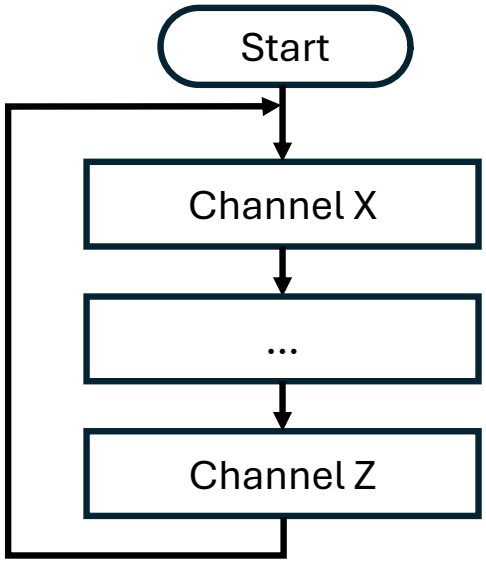


ADC Conversion Mode

Scan Mode: The ADC module can perform sampling and conversion on a set of pre-defined input channels in a round-robin fashion.



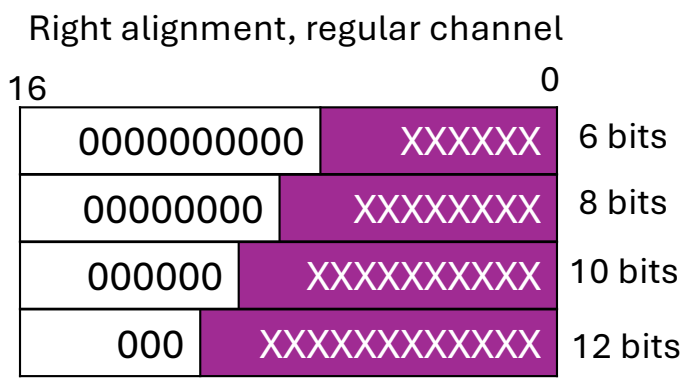
Scan with Single Conversion

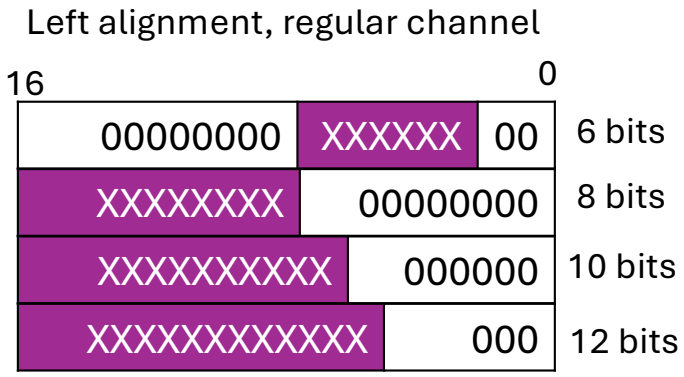


Scan with Continuous Conversion

ADC Data Alignment

- Software can change the ADC resolution (12, 10, 8, or 6 bits)
- Each ADC data register has 16 bits





Acknowledgements

- These lecture slides were adapted from the University of Michigan's EECS 373 course (many thanks to Alanson Sample!)

- These lecture slides also leverage materials from the following books:
  - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C
    - Yifeng Zhu
  - The Definitive Guide to ARM Cortex – M3 and Cortex – M4 Processors
    - Joseph Yiu
  - Introduction to ARM Cortex – M Microcontrollers, Embedded Systems
    - Jonathan W. Valvano